

Introduction:

This time, the focus was on implementing policy-based reinforcement learning, specifically through the implementation of A2C and PPO, to gain a deeper understanding of these methods. During the process, I observed that PPO is generally more effective and easier to train compared to A2C. I also explored various training strategies—such as advantage normalization—and conducted numerical comparisons to evaluate their performance.

Implementation:

1. How do you obtain the stochastic policy gradient and the TD error for A2C?

Stochastic Policy Gradient:

$$J_{\text{actor}}(\theta) = \mathbb{E}_{s \sim d_{\mu}^{\pi_{\theta}}, a \sim \pi_{\theta}(\cdot|s)} [\log \pi_{\theta}(a|s) \cdot A^{\pi_{\theta}}(s, a)] \approx \frac{1}{|B|} \sum_{(s, a, \hat{A}_w(s, a)) \in B} \log \pi_{\theta}(a|s) \hat{A}_w(s, a),$$

```
advantage = (td_target - values).detach()

_, dist = self.actor(states)

log_probs = dist.log_prob(actions).sum(-1, keepdim=True)
entropy = dist.entropy().mean()

policy_loss = -(log_probs * advantage).mean() - self.entropy_weight * entropy
```

To compute the stochastic policy gradient, we first pass the current state through the actor network to obtain the action probability distribution $\pi_{\theta}(a | s)$. We then use the actual action taken to extract the corresponding probability from this distribution.

This probability is used to compute the stochastic policy gradient by multiplying with the advantage (minus td_targets with values)

TD error:

$$\mathbb{E}_{(s,a,r,s')} \left[(r + \gamma V_w(s') - V_w(s))^2 \right]$$

```
with torch.no_grad():
    td_target = rewards + self.gamma * self.critic(next_states) * mask
    values = self.critic(states)
    value_loss = F.mse_loss(values, td_target)
```

We want the Critic to tell us, “Given the current state, how much cumulative reward can we expect from now on?”

To achieve this, we first compute the TD Target, which serves as an estimated answer for the Critic to learn from. Then, we obtain the current value prediction from the Critic.

By subtracting the predicted value from the TD Target, we get the TD Error, which reflects how far the Critic’s current prediction is from the target.

2. How do you implement the clipped objective in PPO?

```
loss1 = ratio * sampled_advantages
loss2 = torch.clamp(ratio, 1.0 - self.epsilon, 1.0 + self.epsilon) * sampled_advantages
actor_loss = -torch.min(loss1, loss2).mean()
entropy = dist.entropy().sum(-1).mean()
actor_loss -= self.entropy_weight * entropy
#print(loss1.shape, loss2.shape, actor_loss.shape, entropy.shape)
```

In PPO, we use the clipped objective to limit the size of each policy update, preventing overly large changes that could cause performance collapse. This helps stabilize training.

In addition to computing the loss using the policy probability ratio, we also provide a clipped version of the ratio to compare against. If the update is too large, we use the clipped loss (loss2); otherwise, if the update is within an acceptable range, we use the original loss (loss1), indicating that the current policy change is acceptable.

3. How do you obtain the estimator of GAE?

```
def compute_gae(
    next_value: list, rewards: list, masks: list, values: list, gamma: float, tau: float) -> List:
    """Compute gae."""

    #####TODO#####
    values = values + [next_value]
    gae = 0
    gae_returns = []
    for step in reversed(range(len(rewards))):
        delta = rewards[step] + gamma * values[step + 1] * masks[step] - values[step]
        gae = delta + gamma * tau * masks[step] * gae
        gae_returns.insert(0, gae + values[step])
    #####
    return gae_returns
```

We first append the next state's value (i.e., next_value) to the end of the values list. Then, we compute the full GAE by iterating backward from the last step. At each step, we calculate the temporal difference error delta, and then use it along with the GAE from the next step to compute the current GAE. This recursive calculation uses gamma to discount future rewards and tau to control the smoothing effect. If an episode has ended at a certain step (mask[step] = 0), it prevents the GAE from propagating beyond that point. Finally, all GAE values are collected and returned.

4. How do you collect samples from the environment?

```
def select_action(self, state: np.ndarray) -> np.ndarray:
    if isinstance(state, np.ndarray):
        state = state.squeeze()
    else:
        state = np.array(state).squeeze()
    state_tensor = torch.FloatTensor(state).to(self.device)
    #print(state_tensor.shape)

    with torch.no_grad():
        action, dist = self.actor(state_tensor)
        log_prob = dist.log_prob(action).sum(dim=-1)
    if not self.is_test:
        with torch.no_grad():
            value = self.critic(state_tensor)
        self.states.append(state_tensor.unsqueeze(0)) # [1, obs_dim]
        self.actions.append(action.unsqueeze(0)) # [1, act_dim]
        self.values.append(value.unsqueeze(0)) # [1, 1]
        self.log_probs.append(log_prob.unsqueeze(0)) # [1, 1]
    select_action = dist.mean.cpu().detach().numpy() if self.is_test else action.cpu().numpy()
    return select_action

def step(self, action: np.ndarray) -> Tuple[np.ndarray, float, bool]:
    next_state, reward, terminated, truncated, _ = self.env.step(action)
    done = terminated or truncated
    if not self.is_test:
        self.rewards.append(torch.tensor([reward], dtype=torch.float32, device=self.device))
        self.masks.append(torch.tensor([1 - done], dtype=torch.float32, device=self.device))
    return np.expand_dims(next_state, axis=0), reward, done
```

```
idx = indices[i:i + self.batch_size]
state = states[idx]
action = actions[idx]
return_ = returns[idx].squeeze(-1)
advantage = advantages[idx].squeeze(-1)
old_log_prob = log_probs[idx]
```

```

def select_action(self, state: np.ndarray) -> np.ndarray:
    """Select an action from the input state."""
    state = torch.FloatTensor(state).to(self.device)
    action_raw, dist = self.actor(state)

    low, high = self.env.action_space.low[0], self.env.action_space.high[0]

    if self.is_test:
        selected_action = dist.mean.clamp(low, high)
    else:
        selected_action = action_raw.clamp(low, high)

    if not self.is_test:
        self.transition = [state, action_raw.detach()]

    return selected_action.cpu().detach().numpy()

def step(self, action: np.ndarray) -> Tuple[np.ndarray, np.float64, bool]:
    """Take an action and return the response of the env."""
    next_state, reward, terminated, truncated, _ = self.env.step(action)
    done = terminated or truncated

    if not self.is_test:
        n_s = torch.FloatTensor(next_state).to(self.device)
        train_reward = (reward + 8.1) / 8.1
        self.transition.extend([train_reward, n_s, done])

    return next_state, reward, done

transition = self.buffer.sample(size)
states, actions, rewards, next_states, dones = zip(*transition)
states = torch.stack(states).float().to(self.device)
actions = torch.stack(actions).float().to(self.device)
next_states = torch.stack(next_states).float().to(self.device)
rewards = torch.tensor(rewards).float().unsqueeze(1).to(self.device)
dones = torch.tensor(dones).float().unsqueeze(1).to(self.device)
mask = 1.0 - dones

```

Both A2C and PPO implement a similar structure:

At each step, after selecting an action based on the current state, we compute the corresponding action probability and critic value, then append these along with the state to a buffer.

After executing the action in the environment via `step()`, we store the resulting next state, reward, and done flag into the buffer as well.

Later, during model updates, we sample a fixed number of these stored experiences based on the defined `batch_size` and use them as the basis for updating the model.

5. How do you enforce exploration (despite that both A2C and PPO are on-policy RL methods)?

```

def forward(self, state: torch.Tensor) -> torch.Tensor:
    """Forward method implementation."""

    #####TODO#####
    output = self.network(state)
    loc = F.tanh(self.mu_layer(output)) * 2
    std = torch.clamp(self.log_std.exp(), 1e-3, 5)
    dist = torch.distributions.Normal(loc, std)
    action = dist.rsample()
    #print(std.item())
    #####

    return action, dist

```

In both A2C and PPO, I calculate the mean and standard deviation first, then use

a Normal distribution to represent the policy's probability distribution. By maintaining a certain level of randomness—ensuring that the standard deviation does not approach zero—the policy samples actions stochastically within the action space, thereby enabling effective exploration.

```
policy_loss = -(log_probs * advantage).mean() - self.entropy_weight * entropy
```

Another approach is to encourage exploration by adding an entropy bonus to the actor_loss. This promotes more diverse action selection, preventing the policy from becoming too deterministic too early.

6. Explain how you use Weight & Bias to track model performance and the loss values (including actor loss, critic loss, and the entropy).

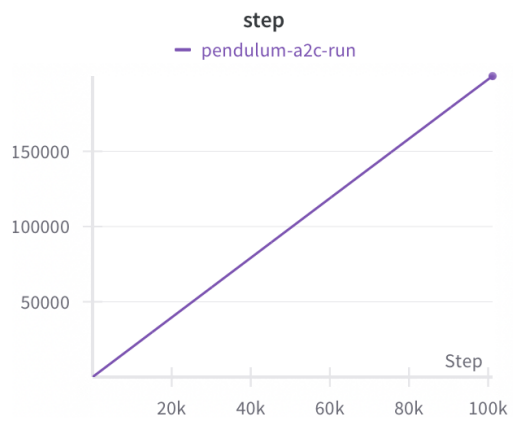
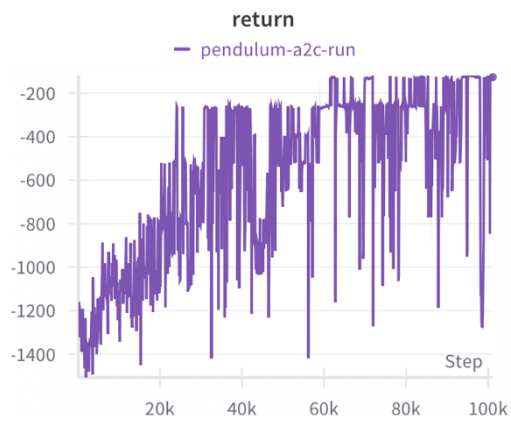
You can observe the results using a similar approach as in Lab 5. Notably, in this implementation, I additionally recorded log_mean and log_std, which helped me in the early stages to verify whether the Actor's forward pass was implemented correctly. This prevented issues such as log_std = 0, which would remove the policy's ability to adapt its exploration strength, potentially leading to ineffective training. Furthermore, I monitored the change in loss to ensure proper convergence of the model, and I used the trend of the returns to check whether the training performance improved compared to before the modifications.

```
wandb.log({"log_mean": log_probs.mean().item(),
          "log_std": log_probs.std().item(),
          "advantage/mean": advantage.mean().item(),
          "advantage/std": advantage.std().item()
        })
```

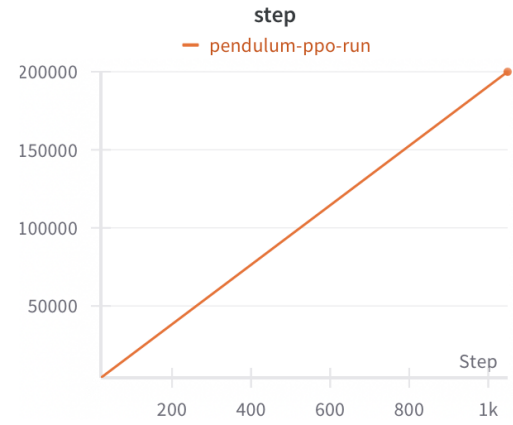
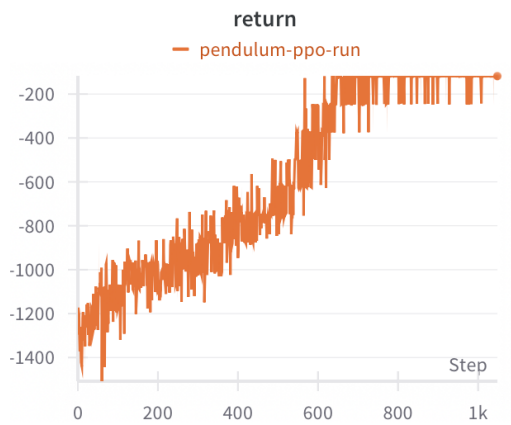
Analysis and discussions :

Training Curve:

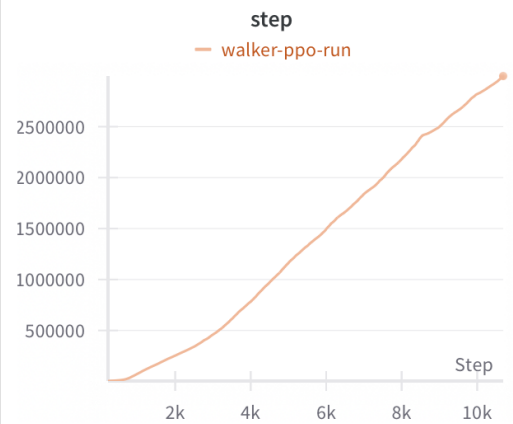
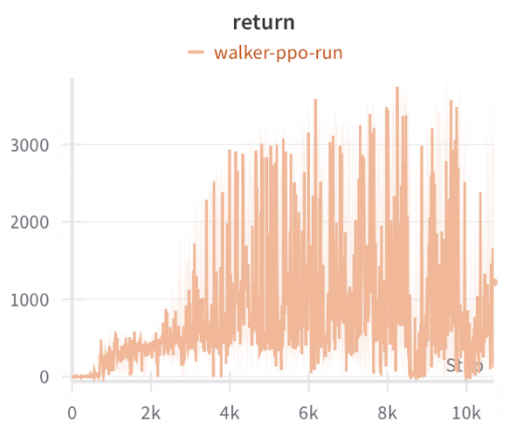
A2C pendulum:

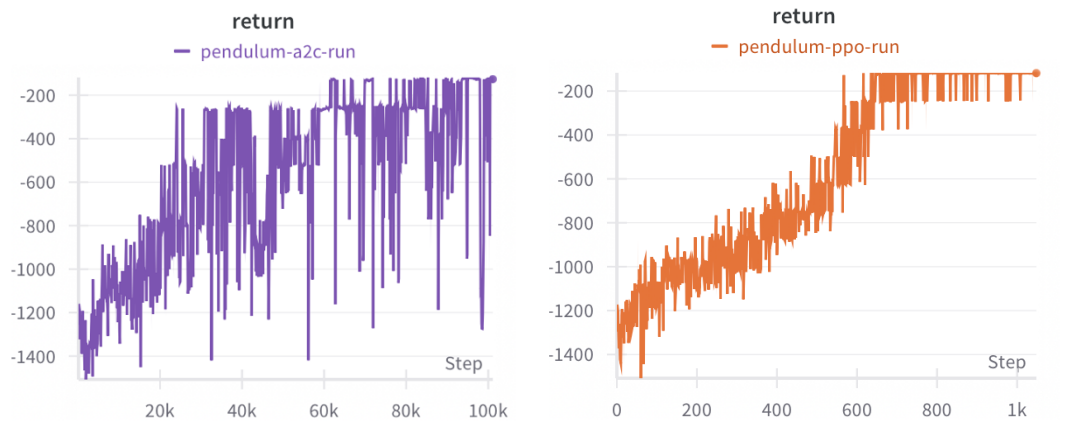


PPO pendulum:



PPO walker:





Compare PPO and A2C:

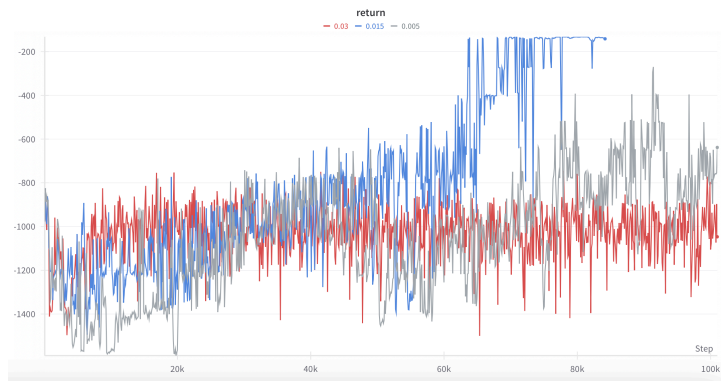
From the plot, we can observe that both A2C and PPO reach their peak performance around a similar proportion of training steps. However, since PPO applies a clipping mechanism on the policy update to restrict large deviations in action probabilities, its learning process is generally more stable — showing a steady increase in performance and maintaining it within a relatively consistent range.

In contrast, A2C tends to be more sensitive to certain factors (such as entropy weight or other hyperparameters), often resulting in significant fluctuations during training. In my own experiments during the lab, I also noticed that PPO converged more easily, while A2C was much more sensitive to hyperparameter tuning, making it harder to stabilize.

Experiment:

1. Different entropy weight:

A2C:



We can observe that different entropy weights have a significant impact on A2C:

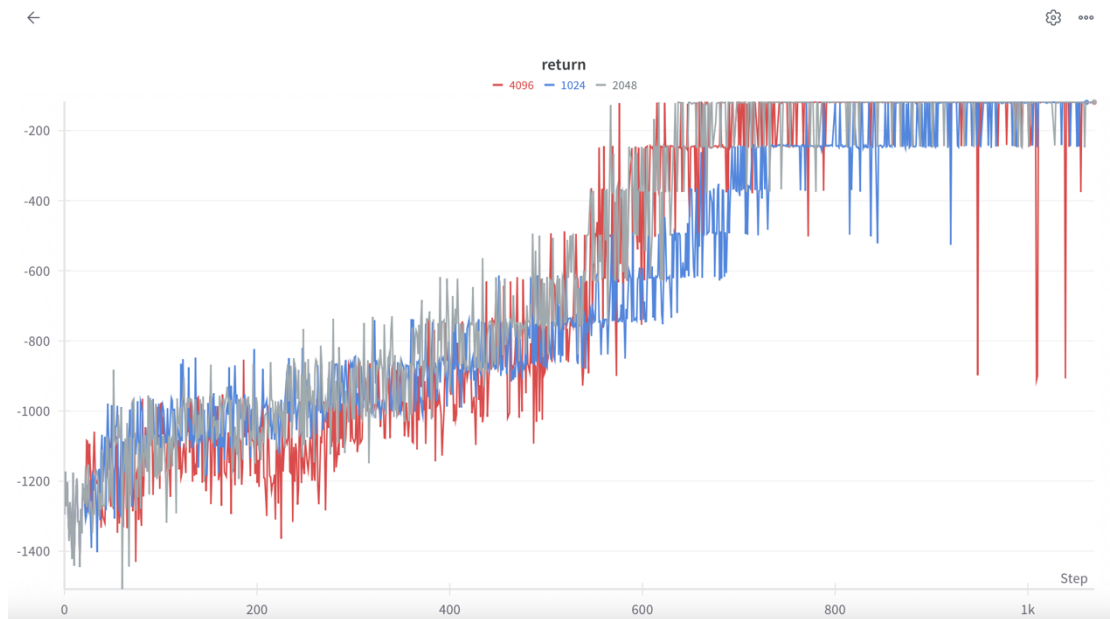
If the entropy weight is set too high, the agent may explore excessively, resulting in overly random behavior that hinders stable learning and leads to lower scores.

On the other hand, if the entropy weight is too low, the agent's actions may become too constrained, reducing exploration, and potentially causing it to get stuck in local optima.

In summary, A2C is highly sensitive to the entropy weight. Choosing an appropriate value is crucial for balancing exploration and exploitation, which is key to successful training.

2. The Different batch size:





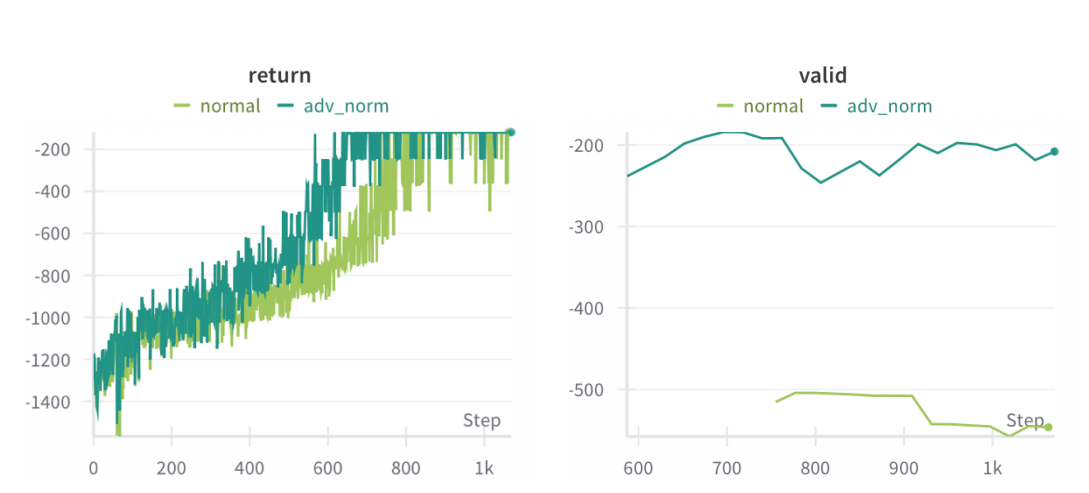
I found that in PPO, different batch sizes have a noticeable impact on training performance. While the overall training trends appear similar across batch sizes, it's evident that using batch sizes of 1024 and 4096 leads to more fluctuation during training compared to 2048 — with 4096 showing the most instability.

Looking at the validation scores, the model trained with a batch size of 2048 tends to converge more steadily and reaches a higher performance, whereas both 1024 and 4096 exhibit larger variations.

This suggests that using a batch size that is too small can introduce high variance in sampled data, making training less stable. On the other hand, using an overly large batch size may cause updates to become too conservative, slowing down learning. Therefore, selecting an appropriate batch size is a critical factor that should not be overlooked during training.

3. Other Training Strategy: Advantage Normalization

After computing the advantages, I applied normalization to make them have a mean of 0 and a standard deviation of 1. This helps prevent extremely large or small values that could cause gradient explosion or vanishing, resulting in more stable policy updates.



We observed that after implementing advantage normalization, the training process became more stable and converged more easily compared to the unnormalized version. The learning curve showed less fluctuation, and the validation performance improved significantly, indicating that advantage normalization helps stabilize training and enhances policy learning efficiency.

4. command & the score record of the PPO_walker:

score record of the PPO_walker: When you running a code , it will write the records into the new file and you can see in it.

```
From 0 ~ 1000000: Best Score: 2734.695982691089 Best Step: 980315
From 1000000 ~ 1500000: Best Score: 2962.6137177679702 Best Step: 1052100
From 1500000 ~ 2000000: Best Score: 2845.017026285398 Best Step: 1540737
From 2000000 ~ 2500000: Best Score: 2849.5315095945134 Best Step: 2370832
From 2500000 ~ 3000000: Best Score: 2743.2032186937304 Best Step: 2732177
```

Training command:

```
python ppo_pendulum.py
```

```
python ppo_walker.py
```

```
python a2c_pendulum.py
```

Testing:

```
python a2c_pendulum.py --test --seed 245
```

```
python ppo_pendulum.py --test --seed 5
```

```
python ppo_walker.py --test
```